COMS/CPRE 4540/5540 Assignment 2

RPC-Based Shared Data-Space System

Due: Saturday, Oct 03, 2026

Points: 30

Regarding AI Usage

The design and coding of this assignment should be done by you, because the goal of this assignment is not just to deliver a working system, but also more importantly for you to understand the principle of RPC and its application. AI tools may be used to help in understanding related concepts and/or resolving programming issues.

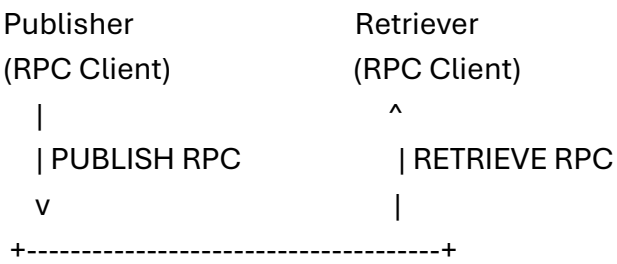
In the case that you have used AI tools in this assignment, state in your README file concisely: (1) what specific AI tools are used and (2) how they are used.

---

1. Objective

In this assignment, you will develop a simple distributed **shared data-space system** using Remote Procedure Call (RPC).

A shared data space provides an indirect mechanism for communication and coordination. Instead of communicating directly with each other, processes place data into a shared space and other processes retrieve data from that space. Your system consists of three types of processes:



```
|    Broker (RPC Server)          |
|    Shared Data Space            |
 +-----------------------------------+
```

The **Broker is the RPC server**. It maintains the shared data space in memory and exports remote procedures through RPC.

There are two types of RPC clients: a **Publisher** publishes an event to the Broker; a **Retriever** requests from the Broker all currently stored events having a specified topic. A published event has the form:

(topic, value)

For example:

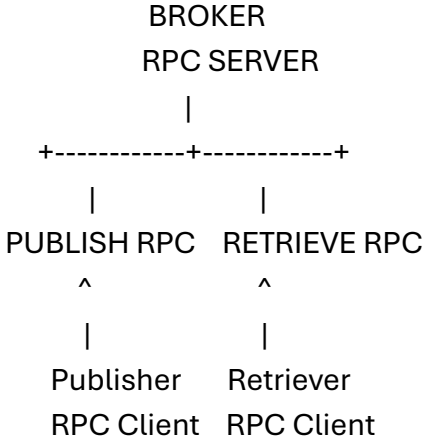
("intrusion", 5)
("temperature", 95)
("intrusion", 8)

The Broker stores published events until the Broker terminates. Retrieving an event does **not** remove it from the shared data space.

You must use **ONC/Sun RPC and rpcgen, along with C/C++** to implement communication between the Publisher/Retriever clients and the Broker.

2. System Architecture

The Broker is the only RPC server in this assignment. Both Publishers and Retrievers are RPC clients.



Publishers and Retrievers do not communicate directly with each other. For example:

Publisher A ---- publish("intrusion", 5) ----> Broker

Publisher A terminates.

... later ...

Retriever B ---- retrieve("intrusion") ------> Broker
|
Retriever B <---------- [5] ----------+

Thus, the Publisher and Retriever do not need to execute at the same time, as long as the Broker remains running and retains the published event.

3. Events and Shared Data Space

Each event contains topic and value, where:

- topic is a string of at most 32 characters; and
- value is an integer.

Topics are matched using exact string matching.

The Broker initially contains no events. After the following publications:

- publish("intrusion", 5)
- publish("temperature", 95)
- publish("intrusion", 8)
- publish("disk_usage", 90)

the Broker's shared data space conceptually contains:

Topic	Value
intrusion	5
temperature	95
intrusion	8
disk_usage	90

The Broker should support an arbitrary number of events, subject to available memory. Do not use a fixed-size array that artificially limits the total number of stored events.

4. RPC Procedures Provided by the Broker

The Broker must export two remote procedures:

- PUBLISH
- RETRIEVE

4.1 PUBLISH

Conceptually, publish(topic, value) adds a new event to the shared data space. For example, publish("intrusion", 5) causes: ("intrusion", 5) to be stored by the Broker. The Broker should return a status indicating whether the operation succeeded. A successful publication does not depend on whether any Retriever currently exists.

4.2 RETRIEVE

Conceptually, retrieve(topic) returns the values of **all currently stored events** whose topic exactly matches the requested topic. Suppose the shared data space contains ("intrusion", 5), ("temperature", 95), ("intrusion", 8), ("disk_usage", 90) and ("intrusion", 12). Then retrieve("intrusion") should return 5, 8 and 12; while retrieve("temperature") should return 95. If no matching event exists, an empty list should be returned.

Retrieval is non-destructive

RETRIEVE does **not** remove events from the shared data space. Therefore: retrieve("intrusion") and retrieve("intrusion") should return the same values unless new matching events have been published between the two requests. For this reason, the client performing this operation is called a **Retriever**, rather than a Consumer.

5. Publisher — RPC Client

The Publisher is an **RPC client**. It should be executed as:

- publisher <broker-host> <topic> <value>

For example:

- ./publisher pyrite-n1.cs.iastate.edu intrusion 5

The Publisher should:

1. process and validate its command-line arguments;
2. establish an RPC connection to the Broker;
3. invoke the Broker's PUBLISH remote procedure;
4. report the result; and
5. terminate.

Example:

$ ./publisher pyrite-n1.cs.iastate.edu intrusion 5

Should publish event (topic=intrusion, value=5).

The Publisher does not need to know whether any Retriever exists or where Retrievers are located.

6. Retriever — RPC Client

The Retriever is also an **RPC client**.

It should be executed as:

- retriever <broker-host> <topic>

For example:

./retriever pyrite-n1.cs.iastate.edu intrusion

The Retriever should:

1. process and validate its command-line arguments;
2. establish an RPC connection to the Broker;
3. invoke the Broker's RETRIEVE remote procedure;
4. receive a variable-length result;
5. print all returned values; and
6. terminate.

For example:

$ ./retriever pyrite-n1.cs.iastate.edu intrusion

Should report events for topic 'intrusion': 3, 5, 8, and 12

If no matching events exist:

$ ./retriever pyrite-n1.cs.iastate.edu network

Should report the number of events for topic 'network' is 0.

7. Example Execution

Start the Broker on pyrite-n1.cs.iastate.edu in one window:

$ ./broker

On the same Pyrite node but different window:

$ ./publisher pyrite-n1.cs.iastate.edu intrusion 5
Published: topic=intrusion, value=5

$ ./publisher pyrite-n1.cs.iastate.edu temperature 95
Published: topic=temperature, value=95

$ ./publisher pyrite-n1.cs.iastate.edu intrusion 8
Published: topic=intrusion, value=8

The Broker now conceptually contains:

("intrusion", 5)
("temperature", 95)
("intrusion", 8)

A Retriever can then execute on the same machine:

$ ./retriever pyrite-n1.cs.iastate.edu intrusion
Report events for topic 'intrusion': 2, 5, and 8

Executing the same Retriever again:

$ ./retriever pyrite-n1.cs.iastate.edu intrusion

should again report events for topic 'intrusion': 2, 5, and 8 because retrieval does not remove events.

8. RPC Programming Guidance

You must use **ONC/Sun RPC and rpcgen, along with C/C++**.

The following material provides guidance on RPC programming and data representation. It does not specify the complete implementation; you are responsible for designing and implementing your Broker, Publisher, and Retriever.

8.1 RPC Interface Definition

RPC arguments and return values must be represented using types understood by RPC/XDR.

An event can, for example, be represented in an RPC interface as:

```
const MAX_TOPIC = 32;
struct event {
   string topic<MAX_TOPIC>;
   int value;
};
```

Conceptually, your Broker exports:

```
PUBLISH(event) -> status
RETRIEVE(topic) -> list of integers
```

You must assign appropriate program, version, and procedure numbers in your .x file.

Running rpcgen on the .x file generates the C data structures and RPC/XDR routines needed by the client and Broker.

8.2 RPC Supports Only One Argument to a Remote Procedure

An RPC procedure formally receives one argument. If a remote procedure logically requires several arguments, combine them into a structure. For example, the logical operation:

```
publish(topic, value)
```

can be represented as:

```
struct event {
   string topic<MAX_TOPIC>;
   int value;
};
```

The Publisher constructs an event and passes that structure as the RPC argument. This

technique can be used whenever multiple logical arguments need to be passed through RPC.

8.3 Strings

RPC/XDR supports strings directly. For example:

string topic<32>;

declares a variable-length string having a maximum length of 32 characters.

rpcgen translates an RPC string into a C char *.

Therefore, generated C code may contain something conceptually similar to:

char *topic;

Remember that the actual character data must remain valid while the RPC call is being made.

9. Variable-Length Data in RPC

The RETRIEVE operation presents an important RPC programming issue. The Broker cannot know in advance how many events will match a topic. For example:

retrieve("intrusion")

might return:

[]

or:

[5]

or:

[5, 8, 12, 20, 31, ...]

Therefore, the return value cannot conveniently be represented by a fixed-size C array. RPC/XDR supports **variable-length arrays** for this purpose.

9.1 Declaring a Variable-Length Array

In an RPC .x file, a variable-length array can be declared using:

typedef int value_list<>;

The empty angle brackets indicate that the array has variable length.

You could alternatively specify a maximum size:

typedef int value_list<100>;

but this assignment requires the number of stored events to be limited only by available memory, so an unbounded RPC array is more appropriate:

typedef int value_list<>;

9.2 How rpcgen Represents a Variable-Length Array

C itself does not represent an RPC variable-length array using a simple array variable. For:

typedef int value_list<>;

rpcgen generates a C structure conceptually similar to:

```
typedef struct {
   u_int value_list_len;
   int *value_list_val;
} value_list;
```

There are therefore two important fields:

value_list_len

contains the number of elements, while:

value_list_val

points to the dynamically allocated array containing those elements.

For example, the RPC result:

[5, 8, 12]

is represented conceptually as:

value_list_len = 3

value_list_val
        |
        v
+----+----+----+
| 5 | 8 | 12 |
+----+----+----+

10. Constructing a Variable-Length Result at the Broker

Suppose the Broker receives:

retrieve("intrusion")

and determines that three stored events match: 5, 8, and 12. The Broker's RPC implementation must construct a result containing:

length = 3
values = [5, 8, 12]

Conceptually:

```
result.value_list_len = 3;
result.value_list_val = malloc(3 * sizeof(int));
result.value_list_val[0] = 5;
result.value_list_val[1] = 8;
result.value_list_val[2] = 12;
```

The actual field names depend on the declarations in your .x file.

For no matching events, the result should represent an empty array:

```
result.value_list_len = 0;
result.value_list_val = NULL;
```

Do **not** return a pointer to a local C array such as:

int values[100];

that ceases to be valid when the function returns.

You must also manage dynamically allocated result memory correctly across repeated RPC calls.

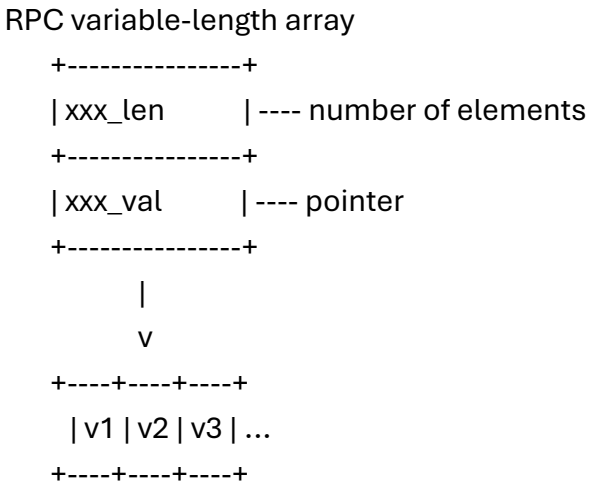
11. Processing a Variable-Length Result at the Retriever

After calling RETRIEVE, the Retriever receives the RPC-generated structure.

Conceptually, it can process the returned values as:

```
for (i = 0; i < result->value_list_len; i++) {
   printf("%d\n", result->value_list_val[i]);
}
```

Therefore, students should remember:

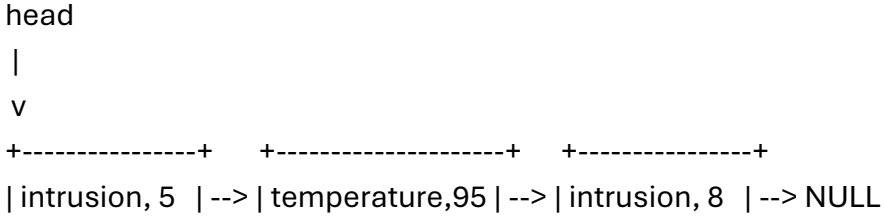


The exact xxx_len and xxx_val names are generated from the name used in the .x file.

12. Suggested Broker Data Structure

The Broker must maintain an arbitrary number of events.

A linked list is one possible implementation:



```
+----------------+     +--------------------+     +----------------+
```

A possible C representation is:

```
struct event_node {
   char *topic;
   int value;
   struct event_node *next;
};
```

This is only a suggestion. You may use another dynamically sized data structure.

When implementing PUBLISH, the Broker adds an event to the data structure.

When implementing RETRIEVE, the Broker searches the data structure for all matching topics and constructs the variable-length RPC result.

13. RPC Client Programming

Both Publisher and Retriever are RPC clients.

A client generally performs the following steps:

```
command-line arguments
      |
      v
create RPC client connection
      |
      v
prepare RPC argument
      |
      v
invoke generated RPC client stub
      |
      v
process returned result
      |
      v
```

destroy RPC client connection

For example, a client normally creates an RPC handle using an operation such as:

```
CLIENT *clnt;

clnt = clnt_create(
  broker_hostname,
  PROGRAM_NUMBER,
  VERSION_NUMBER,
  "tcp"
);
```

Always check whether creation of the RPC client succeeds.

If it fails, report an appropriate error rather than continuing with an invalid client handle.

The generated RPC client stubs should then be used to invoke PUBLISH or RETRIEVE.

14. Broker RPC Programming

The Broker is the RPC server.

Its responsibilities include:

1. registering its RPC service;
2. receiving RPC requests;
3. executing the requested operation;
4. maintaining the shared data space;
5. constructing RPC return values; and
6. remaining available for subsequent requests.

The Broker must not terminate after processing one Publisher or Retriever.

15. Generated RPC Files

Depending on the rpcgen options used, RPC development typically involves:

```
dataspace.x
  |
  | rpcgen
  v
```

```
+------------------------+
| dataspace.h            |
| dataspace_clnt.c       |
| dataspace_svc.c        |
| dataspace_xdr.c        |
+------------------------+
```

Conceptually:

- dataspace.h contains generated declarations;
- dataspace_clnt.c contains client-side RPC stubs;
- dataspace_svc.c contains server-side RPC dispatch code; and
- dataspace_xdr.c contains serialization/deserialization routines.

You should add your own implementation code for the Broker, Publisher, and Retriever as appropriate. Do not manually serialize RPC arguments into network byte streams. The generated XDR routines perform the serialization for you.

16. Error Handling

Your programs should reasonably handle:

- incorrect command-line arguments;
- invalid topic names;
- invalid integer values;
- inability to contact the Broker;
- RPC failures; and
- memory-allocation failures.

An invalid request should not cause the Broker to terminate.

17. Compilation and Execution Requirements

The assignment must be implemented in **C/C++ using ONC/Sun RPC**.

Your submission must contain a Makefile.

Running:

make

should produce:

- broker
- publisher
- retriever

Your programs must compile and run on the same or different nodes of:

pyrite.cs.iastate.edu

Do not hard-code Broker hostnames or IP addresses.

18. Submission

Submit one ZIP package containing:

- RPC interface (.x) file;
- Broker source code;
- Publisher source code;
- Retriever source code;
- other source files necessary for compilation;
- Makefile; and
- README.

The README should briefly explain:

- how to compile the programs;
- how to start the Broker;
- how to run a Publisher;
- how to run a Retriever; and
- any implementation assumptions.

19. Score Allocation

Requirement	Points
Programs compile and run on Pyrite	10
RPC/IDL definition and RPC usage	3
Correct PUBLISH implementation	4
Correct RETRIEVE implementation	4
Dynamic number of stored/returned events	3

Publisher and Retriever user interfaces	2
Error handling	2
Makefile, organization, and comments	2
Total	**30**